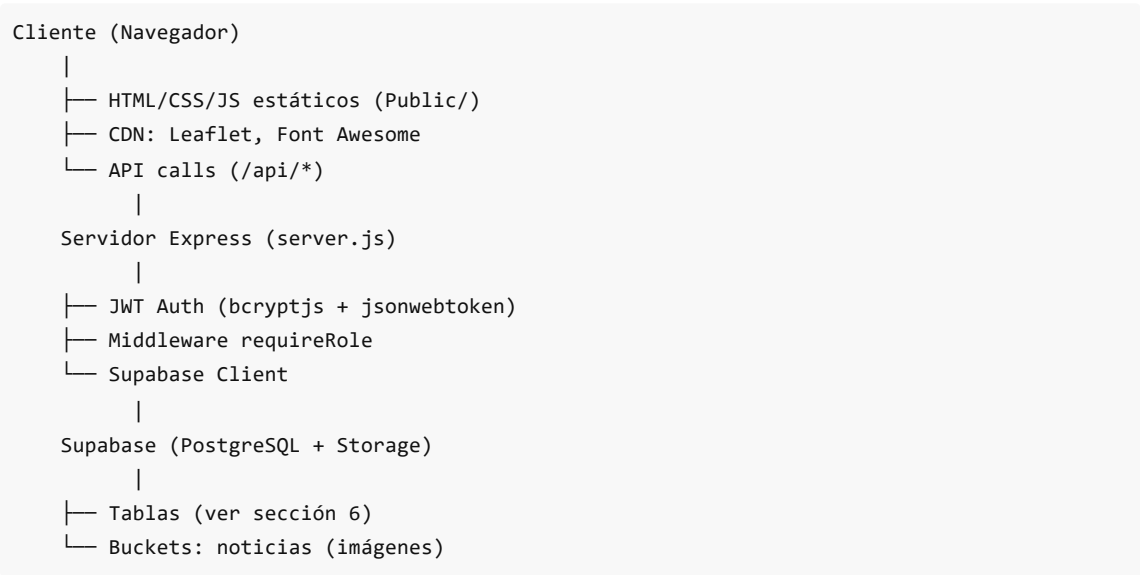


Manual Técnico — Pitahaya Biotec

Índice

- 1. [Arquitectura del Sistema](#)
 - 2. [Stack Tecnológico](#)
 - 3. [Estructura del Proyecto](#)
 - 4. [Configuración del Entorno](#)
 - 5. [Autenticación y Autorización](#)
 - 6. [Base de Datos](#)
 - 7. [Despliegue](#)
 - 8. [API Endpoints](#)
 - 9. [Mantenimiento](#)
 - 10. [Solución de Problemas](#)
-

1. Arquitectura del Sistema



Hosting: Vercel (serverless Node.js) **Base de datos:** Supabase PostgreSQL **Storage:** Supabase Storage (imágenes)

2. Stack Tecnológico

Componente	Tecnología	Versión
Backend	Node.js, Express	22.x / 4.x
Base de datos	PostgreSQL (Supabase)	15.x
Autenticación	JWT (jsonwebtoken) + bcryptjs	9.x / 2.x
Frontend	HTML5, CSS3, JS vanilla	—
Mapas	Leaflet (CDN)	1.x

Iconos	Font Awesome 6 (CDN)	6.5
Chatbot IA	Groq API (Llama 3.3) + KB local	—
Storage	Supabase Storage	—
Despliegue	Vercel (@vercel/node)	—

Dependencias (package.json)

```
{
  "dependencies": {
    "@supabase/supabase-js": "^2.x",
    "bcryptjs": "^2.x",
    "cors": "^2.x",
    "dotenv": "^16.x",
    "express": "^4.x",
    "jsonwebtoken": "^9.x",
    "pg": "^8.x"
  }
}
```

3. Estructura del Proyecto

```
Pitahaya-main/
├─ server.js           # Servidor Express (803 líneas)
├─ package.json
├─ vercel.json         # Configuración de despliegue Vercel
├─ .env               # Variables de entorno (Supabase)
├─ .env.local         # Token Vercel OIDC
├─ supabase_setup.sql  # DDL tablas faltantes
├─ setup.js           # DDL inicial + seed data
├─ setup2.js          # DDL noticias, likes, semillas
├─ setup3.js          # Migración: columna estado en pedidos
├─ setup4.js          # Migraciones adicionales
├─ MANUAL_USUARIO.md  # Manual de usuario
├─ MANUAL_TECNICO.md  # Manual técnico (este documento)
├─
├─ Public/
│  ├─ index.html      # Login
│  ├─ registro.html   # Registro
│  ├─ inicio.html     # Dashboard principal
│  ├─ biotec-dashboard.html # Atlas de la Pitahaya
│  ├─ noticias.html   # CRUD noticias
│  ├─ tipos.html      # CRUD variedades
│  ├─ fao.html        # FAO Passport
│  ├─ investigadores.html # Directorio investigadores
│  ├─ investigador-perfil.html # Perfil profesional
│  └─ publicaciones.html # Publicaciones científicas
```

```

|   ├── semillas.html      # Solicitud de semillas
|   ├── bioproductos.html  # Catálogo bioproductos
|   ├── pedidos.html       # Pedidos
|   ├── mapa.html          # Mapa interactivo
|   ├── fertirriego.html   # Calculadora riego
|   ├── soporte.html       # Soporte técnico
|   ├── admin.html         # Panel administración
|   ├── perfil.html        # Perfil de usuario
|   |
|   ├── css/
|   |   ├── tokens.css     # Variables CSS (colores, sombras, radios)
|   |   ├── styles.css     # Estilos globales
|   |   ├── dashboard-style.css
|   |   └── biotec-dashboard.css
|   |
|   ├── js/
|   |   ├── script.js      # Lógica login/registro
|   |   ├── nav.js         # Menú dinámico + fetch interceptor JWT
|   |   └── dashboard-script.js # Mapa Leaflet + grid bioproductos
|   |
|   └── img/
|       └── fondoPitahaya.png
|
└── node_modules/

```

4. Configuración del Entorno

4.1 Variables de Entorno (.env)

```

SUPABASE_URL=https://<project>.supabase.co
SUPABASE_SERVICE_KEY=sb_secret_<your_service_key>
SUPABASE_ANON_KEY=sb_publishable_<your_anon_key>
PORT=3000
JWT_SECRET=<your_jwt_secret>

```

La variable `JWT_SECRET` tiene un valor por defecto en `server.js` línea 9:

```
const JWT_SECRET = process.env.JWT_SECRET || '<your_jwt_secret>';
```

4.2 Instalación Local

```

git clone <repo>
cd Pitahaya-main
npm install
cp .env.example .env # configurar credenciales
npm run dev          # nodemon (desarrollo)
npm start            # node (producción)

```

4.3 Verificación

```
curl http://localhost:3000/api/usuarios
# Debe responder con JSON (array de usuarios) o error de conexión a Supabase
```

5. Autenticación y Autorización

5.1 Flujo de Autenticación

```
Login → POST /api/login → JWT { id, role } → localStorage('userToken')
                                     ↓
                               fetch interceptor (nav.js)
                                     ↓
                        Authorization: Bearer <token> en /api/*
```

5.2 Middleware requireRole

```
function requireRole(roles) {
  return async (req, res, next) => {
    // 1. Intentar JWT desde Authorization header
    const auth = req.headers['authorization'];
    if (auth && auth.startsWith('Bearer ')) {
      const decoded = jwt.verify(auth.slice(7), JWT_SECRET);
      req.authedUser = decoded;
      if (!roles.includes(decoded.role))
        return res.status(403).json({ error: 'Permiso denegado' });
      return next();
    }
    // 2. Fallback legacy: usuario_id en body/query/headers
    // ...
  };
}
```

5.3 Roles

Rol	Valor en BD	Permisos CRUD
Administrador	admin	Todos los endpoints
Investigador	investigador	CRUD contenido científico
Técnico	tecnico	CRUD campo + soporte
Productor	productor	Solo lectura + solicitudes

5.4 JWT Fetch Interceptor (nav.js)

Todo fetch a `/api/*` incluye automáticamente:

- Authorization: Bearer <token> si existe userToken en localStorage
- x-usuario-id: <userId> como fallback para usuarios sin JWT

6. Base de Datos

6.1 Esquema

```
-- Tablas creadas con setup.js / supabase_setup.sql:

usuarios (id, nombre, correo, password, role, created_at)
noticias (id, titulo, contenido, imagen_url, fecha, autor_id, created_at)
likes (id, noticia_id, usuario_id, created_at) -- UNIQUE(noticia_id, usuario_id)
comentarios (id, noticia_id, usuario_id, contenido, created_at)
variedades (id, nombre, nombre_cientifico, descripcion, imagen_url,
             beneficios, localidad, produccion, caracteristicas, created_at)
fao_passport (id, codigo, numero_acceso, numero_colecta, genero, especie,
              subtaxa, nombre_comun, nombre_accesion, fecha_adquisicion,
              pais, ubicacion, latitud, longitud, altitud,
              estado_biologico, fuente_colecta, created_at)
publicaciones (id, titulo, autores, revista, tipo, año, doi,
               resumen, archivo_url, created_at)
soporte_mensajes (id, usuario_id, mensaje, respuesta, respondido_por,
                  estado, created_at, responded_at)
logs_actividad (id, usuario_id, accion, tabla, registro_id, created_at)
semillas (id, usuario_id, variedad, cantidad, direccion, estado, created_at)
pedidos (id, usuario_id, producto, cantidad, mensaje, estado, created_at)
bioproductos (id, nombre, descripcion, precio, stock, imagen_url, created_at)
puntos_monitoreo (id, nombre_finca, latitud, longitud, variedad,
                  estado_salud, created_at)
perfiles_investigador (id, usuario_id, foto_url, nombres, apellidos, ...)
investigaciones (id, perfil_id, titulo, descripcion, fecha, ...)
```

6.2 Comandos DDL

Los archivos SQL en la raíz del proyecto contienen las sentencias CREATE TABLE:

Archivo	Tablas
setup.js	usuarios, puntos_monitoreo, bioproductos, pedidos + seed data
setup2.js	noticias, likes, semillas
setup3.js	ALTER TABLE pedidos ADD COLUMN estado
supabase_setup.sql	fao_passport, comentarios, logs_actividad, variedades, publicaciones, soporte_mensajes

6.3 Tablas Pendientes de Crear

Las siguientes tablas pueden no existir en Supabase. Ejecutar en el SQL Editor del Dashboard:

```

-- soporte_mensajes
CREATE TABLE IF NOT EXISTS soporte_mensajes (
  id SERIAL PRIMARY KEY,
  usuario_id INTEGER NOT NULL,
  mensaje TEXT NOT NULL,
  respuesta TEXT,
  respondido_por INTEGER,
  estado TEXT DEFAULT 'abierto',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  responded_at TIMESTAMPTZ
);

-- publicaciones
CREATE TABLE IF NOT EXISTS publicaciones (
  id SERIAL PRIMARY KEY,
  titulo TEXT NOT NULL,
  autores TEXT,
  revista TEXT,
  tipo TEXT DEFAULT 'articulo',
  año INTEGER,
  doi TEXT,
  resumen TEXT,
  archivo_url TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

```

7. Despliegue

7.1 Vercel (Producción)

```
npx vercel --prod
```

El archivo `vercel.json` enruta todo el tráfico a `server.js` :

```

{
  "version": 2,
  "builds": [{ "src": "server.js", "use": "@vercel/node" }],
  "routes": [{ "src": "/*", "dest": "server.js" }]
}

```

7.2 Variables en Vercel

Configurar en Vercel Dashboard > Project Settings > Environment Variables:

Variable	Valor
SUPABASE_URL	https://<project>.supabase.co
SUPABASE_SERVICE_KEY	sb_secret_<your_service_key>

JWT_SECRET	<your_jwt_secret>
------------	-------------------

7.3 Pipeline CI/CD

El proyecto usa Vercel OIDC token (`VERCEL_OIDC_TOKEN`) para despliegues automatizados desde `.env.local` .

8. API Endpoints

8.1 Auth

Método	Ruta	Auth	Descripción
POST	/api/login	No	Iniciar sesión
POST	/api/register	No	Registrar usuario

8.2 Contenido

Método	Ruta	Auth	Descripción
GET	/api/noticias	No	Listar noticias
POST	/api/noticias	admin, investigador, tecnico	Crear noticia
PUT	/api/noticias/:id	admin, investigador, tecnico	Editar noticia
DELETE	/api/noticias/:id	admin, investigador, tecnico	Eliminar noticia

8.3 FAO Passport

Método	Ruta	Auth	Descripción
GET	/api/fao-passport	No	Listar accesiones
POST	/api/fao-passport	admin, investigador, tecnico	Crear accesión
POST	/api/fao-passport/bulk	admin, investigador, tecnico	Importar múltiples
PUT	/api/fao-passport/:id	admin, investigador, tecnico	Editar
DELETE	/api/fao-passport/:id	admin, investigador, tecnico	Eliminar

8.4 Variedades

Método	Ruta	Auth	Descripción
GET	/api/variedades	No	Listar
POST	/api/variedades	admin, investigador, tecnico	Crear
PUT	/api/variedades/:id	admin, investigador, tecnico	Editar
DELETE	/api/variedades/:id	admin, investigador, tecnico	Eliminar

8.5 Publicaciones

Método	Ruta	Auth	Descripción
GET	/api/publicaciones	No	Listar
POST	/api/publicaciones	admin, investigador, tecnico	Crear
PUT	/api/publicaciones/:id	admin, investigador, tecnico	Editar
DELETE	/api/publicaciones/:id	admin, investigador, tecnico	Eliminar

8.6 Soporte

Método	Ruta	Auth	Descripción
POST	/api/soporte	No	Enviar mensaje
GET	/api/soporte/:usuario_id	No	Mensajes del usuario
GET	/api/soporte	No	Todos (admin/técnico)
PUT	/api/soporte/:id	admin, tecnico	Responder

8.7 Mapas

Método	Ruta	Auth	Descripción
GET	/api/puntos	No	Listar puntos
POST	/api/puntos	admin, investigador, tecnico	Crear
PUT	/api/puntos/:id	admin, investigador, tecnico	Editar
DELETE	/api/puntos/:id	admin, investigador, tecnico	Eliminar

8.8 Bioproductos y Pedidos

Método	Ruta	Auth	Descripción
GET	/api/bioproductos	No	Listar
POST	/api/bioproductos	admin, investigador, tecnico	Crear
PUT	/api/bioproductos/:id	admin, investigador, tecnico	Editar
DELETE	/api/bioproductos/:id	admin, investigador, tecnico	Eliminar
POST	/api/pedidos	No	Crear pedido
GET	/api/pedidos/:usuario_id	No	Pedidos del usuario
GET	/api/pedidos	No	Todos (admin)

8.9 Investigadores

Método	Ruta	Auth	Descripción
GET	/api/investigadores	No	Listar perfiles públicos
GET	/api/investigadores/:id	No	Perfil completo
PUT	/api/investigador/perfil	No	Crear/editar perfil propio
POST	/api/investigador/investigaciones	No	Crear investigación

8.10 Utilidades

Método	Ruta	Auth	Descripción
POST	/api/subir-imagen	No	Subir imagen a Storage
POST	/api/chat	No	Chatbot IA
GET	/api/exportar/:tabla	No	Exportar datos (csv/json/txt)

9. Mantenimiento

9.1 Tareas Periódicas

- **Verificar logs de actividad:** Tabla `logs_actividad` registra todas las operaciones CRUD
- **Respalidar base de datos:** Usar herramienta de exportación de Supabase Dashboard
- **Monitorear Storage:** Revisar uso del bucket `noticias` en Supabase Storage

9.2 Agregar Nueva Tabla

1. Agregar CREATE TABLE en `supabase_setup.sql`
2. Ejecutar en Supabase SQL Editor
3. Agregar endpoints en `server.js`
4. Agregar frontend en `Public/`

9.3 Agregar Nueva Página

1. Crear archivo HTML en `Public/`
2. Incluir `<div id="app-nav"></div>` y `<script src="js/nav.js"></script>`
3. Agregar entrada en `sections` dentro de `nav.js`
4. Si requiere protección, verificar `userId` al inicio del script

10. Solución de Problemas

10.1 Error "Base de datos no disponible"

Verificar variables de entorno:

```
echo $SUPABASE_URL
echo $SUPABASE_SERVICE_KEY
```

10.2 Error "No autorizado - usuario no identificado"

Causas:

- El usuario no ha iniciado sesión (falta `userToken` o `userId` en `localStorage`)
- El token JWT expiró (vigencia: 7 días)
- El interceptor de fetch en `nav.js` no se está ejecutando

Solución: Cerrar sesión y volver a iniciar.

10.3 Error 500 en soporte admin

Si `/api/soporte` devuelve 500, puede ser por un JOIN fallido. Verificar que la tabla `soporte_mensajes` exista y tenga las columnas correctas.

10.4 El chat de soporte no envía mensajes

Verificar en consola del navegador (F12):

- `SyntaxError: Illegal return statement` → `return;` fuera de función en `soporte.html`
- `enviarMensaje is not defined` → el script no cargó por el error anterior

Solución: Actualizar el archivo `soporte.html` (ya corregido en producción).

10.5 Las imágenes no se cargan

Verificar:

- Bucket `noticias` existe en Supabase Storage
- Políticas RLS del bucket permiten lectura pública
- La URL almacenada en BD es accesible

10.6 Despliegue falla en Vercel

```
npx vercel logs <deployment-url>
```

Causas comunes:

- Variables de entorno no configuradas en Vercel Dashboard
- Error de sintaxis en `server.js`
- Dependencia faltante en `package.json`

Apéndice A: Supabase Setup

Para inicializar la base de datos desde cero:

1. Ir a Supabase Dashboard > SQL Editor
2. Ejecutar `supabase_setup.sql` (tablas principales)
3. Ejecutar `setup.js` localmente con Node.js (seed data)
4. Crear bucket `noticias` en Storage (política pública)
5. Verificar RLS policies si es necesario

Apéndice B: Estructura del JWT

```
{
  "id": 1,
  "role": "admin",
  "iat": 1700000000,
  "exp": 1700604800
}
```

- Firmado con HMAC-SHA256 usando `JWT_SECRET`
- Expiración: 7 días (`TOKEN_EXPIRY`)
- Se decodifica en `requireRole` middleware